3.2.7 Ask signature

Previously, you learned that when you use `ask` with `Parse(replyTo: ActorRef[Worker.Response])`, the `replyTo` is added for you. To understand why this happens, let's look at the `ask` signature:

```
package akka.actor.typed.scaladsl

trait ActorContext ... {

  ...

   def ask[Req, Res](target: RecipientRef[Req], createRequest:
   ➥ActorRef[Res] => Req)(mapResponse: Try[Res] => T)(implicit
   ➥responseTimeout: Timeout, classTag: ClassTag[Res]): Unit

  ...
}
```

It may seem daunting, so let's take it step by step.

In the previous section, in `context.ask(worker, Worker.Parse)`, `worker` corresponds to the `target` parameter and the `Worker.Parse` corresponds to `createRequest`. The annotations `[Req, Res]` at the beginning of the method are placeholders for the types in this method. For example, for `Req`, the compiler can check whether the type you use in `target` `RecipientRef[Req]` is the same as the one in `createRequest: ActorRef[Res] => Req`. In this example and the previous one, `Req` is `Worker.Command`.

`Worker.Parse` may look like an object, but as you just saw, `createRequest` is a function with the signature